RAPPORT DE PROJET

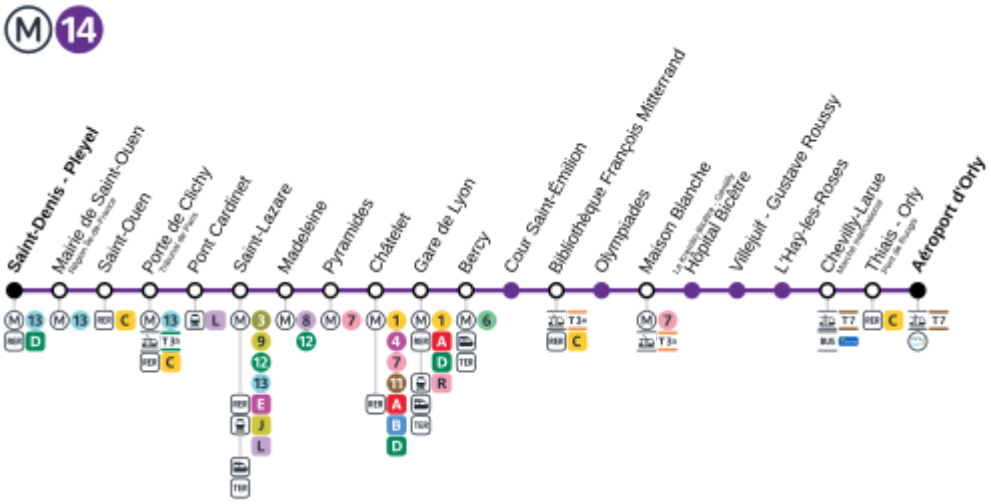
M14-CriticalSystem

Modélisation formelle et simulation d'un système critique ferroviaire

Équipe

Duc Duy • Ostrean • Axel • Alice

Promotion : ING2 – GIA1
Année scolaire : 2025 / 2026



Sommaire

Introduction & Contexte
- 1.1 Présentation générale du projet
- 1.2 Objectifs du projet
- 1.3 Enjeux métier – système critique M14
- 1.4 État de l'art : vérification formelle et réseaux de Petri
- 1.5 Genèse du projet
- 1.6 Outils et technologies

Architecture technique
- 2.1 Vue d'ensemble
- 2.2 Composants principaux
- 2.3 Protocole de communication

Modèle formel – Réseau de Petri
- 3.1 Justification du modèle
- 3.2 Places du réseau
- 3.3 Transitions du réseau
- 3.4 Invariants du réseau
- 3.5 Vue d'ensemble du réseau
- 3.6 Sous-système des portes (PSD)

Vérification des propriétés
- 4.1 Espace d'états atteignables
- 4.2 Propriétés de sûreté (Safety)
- 4.3 Propriétés de vivacité (Liveness)
- 4.4 Propriétés LTL

Répartition des tâches
- 5.1 Organisation générale

Tests & Validation
- 6.1 Stratégie de test
- 6.2 Résultats des tests
- 6.3 Scénarios critiques

Résultats obtenus
- 7.1 Livrables produits
- 7.2 Bilan technique
- 7.3 Reproductibilité des résultats

Conclusion & Perspectives
- 8.1 Bilan global
- 8.2 Limites assumées
- 8.3 Autocritique et limites de fond
- 8.4 Perspectives d'amélioration

Bibliographie

1. Introduction & Contexte

1.1 Présentation générale du projet

M14-CriticalSystem est un projet académique centré sur la modélisation formelle et la simulation d'un système critique inspiré du domaine ferroviaire automatisé (ligne 14 du métro parisien). Le projet étudie un sous-système précis : le contrôle d'accès concurrent de deux trains autonomes sur un tronçon partagé, de la demande d'entrée en canton jusqu'au départ en station avec gestion des portes palières.

Le périmètre est volontairement restreint mais complet :

- 2 trains concurrents
- 1 canton de signalisation partagé
- 1 quai avec portes palières (PSD - Platform Screen Doors ou portes palières en anglais)
- 3 contrôleurs d'arbitrage (SectionController, QuaiController, GestionnairePortes)

Le modèle combine 5 acteurs Akka Typed et un réseau de Petri de 12 places / 12 transitions analysées exhaustivement (20 marquages atteignables, 5 invariants, 0 deadlock). Ce choix permet une analyse formelle complète et démontrable, construite en deux paliers : un socle d'exclusion mutuelle sur le canton, puis une extension PSD ajoutant les propriétés critiques de sûreté des portes.

1.2 Objectifs du projet

#	Objectif	Description
1	État de l'art	Étudier la vérification formelle et les réseaux de Petri appliqués aux systèmes critiques
2	Modélisation Akka	4 acteurs concurrents (Train, SectionController, QuaiController, GestionnairePortes)
3	Réseau de Petri	Construire le réseau étendu (12 places, 12 transitions) représentant le système complet
4	Vérification formelle	Prouver 6 invariants : 3 de ressource + 2 PSD critiques + 1 cohérence par train
5	LTL & Model checking	Formaliser et vérifier 5 propriétés LTL sur le graphe fini (34 Safety + 2 Liveness)
5	Simulation & comparaison	Comparer Akka et Petri sur 3 scénarios critiques, produire une démo web interactive

1.3 Enjeux métier – Sûreté critique M14

Le thème des transports automatisés a été choisi pour sa forte dimension industrielle et ses enjeux de sécurité.

Les principaux enjeux sont :

- gestion de la concurrence entre entités actives
- prévention des conflits d'accès à une ressource partagée
- garantie de sûreté et de vivacité

Dans un système réel, une défaillance peut entraîner des conséquences graves, voire catastrophiques.

1.4 Etat de l'art - Verification formelle & Réseaux de Petri

Vérification formelle

La vérification formelle consiste à utiliser des méthodes mathématiques pour prouver qu'un système respecte ses spécifications.

Elle se distingue des tests classiques car elle analyse **toutes les exécutions possibles**.

Deux grandes catégories de propriétés existent :

- **Sûreté** : rien de mauvais n'arrive
- **Vivacité** : quelque chose de bon finit par arriver

Réseaux de Petri

Les réseaux de Petri (Petri, 1962) sont un formalisme graphique permettant de modéliser des systèmes concurrents.

Ils sont composés de :

- places (états / ressources)
- transitions (événements)
- arcs (relations)

Le marquage représente l'état du système.

Ils permettent de vérifier :

- bornitude
- vivacité
- absence de blocage (deadlock)
- invariants de système

Logique temporelle LTL

La logique LTL (Pnueli, 1977) permet d'exprimer des propriétés temporelles :

- G : toujours
- F : finalement
- X : prochain état
- U : jusqu'à

Exemple :

- G ¬(T1 ∧ T2) → deux trains ne sont jamais sur le canton en même temps

1.5 Genèse du projet

Au début, on a choisi un système de billet pour ce projet mais ça n'a pas été assez critiquable. je (Duc Duy) cherchais donc un autre sujet de projet qui soit à la fois technique, ancré dans une problématique réelle, et compatible avec ce que nous venions d'apprendre en Scala et en programmation concurrente. Aucune des idées initialement proposées au sein de l'équipe ne me convainquait vraiment.

L'élément déclencheur a été un trajet à Paris : j'ai subi un retard significatif sur la ligne 14, première ligne entièrement automatique du métro parisien (mise en service en 1998, sans conducteur, exploitée par la RATP avec un système CBTC). En attendant sur le quai,j'ai commencé à me poser des questions très concrètes : comment deux rames automatiques peuvent-elles partager une même voie sans jamais entrer en collision ? Qui décide quand les portes palières s'ouvrent ? Que se passe-t-il en cas d'incident ? Toutes ces questions renvoyaient précisément aux notions d'exclusion mutuelle, de sections critiques, de synchronisation entre processus concurrents et de propriétés de sûreté, exactement le cœur du programme.

J'ai proposé ce sujet à l'équipe le lendemain. La transposition était directe : un tronçon partagé → une section critique ; deux trains →deux processus concurrents ; les portes palières (PSD) → une ressource exclusive supplémentaire. Le caractère critique du système réel (un bug peut tuer) justifiait à lui seul l'effort de vérification formelle par réseaux de Petri, au-delà des simples tests unitaires. L'équipe a adopté le sujet, et nous avons cadré ensemble le périmètre minimal mais défendable que vous voyez décrit dans ce rapport.

1.6 Outils et technologies

Outil / Technologie	Rôle dans le projet
Scala 2.13	Langage principal – implémentation de toute la logique
Akka Typed	Framework d'acteurs – simulation distribuée du protocole de messages
Réseaux de Petri	Modèle formel : 12 places, 12 transitions, 20 marquages, 40 arcs
ScalaTest + Akka TestKit	Tests unitaires et intégration (suites SectionController, Train, QuaiController, GestionnairePortes, Analyseur)
Demo Web HTML/JS	Simulateur web pour rejouer des traces JSON (nominal, concurrence, violation)
Github	Gestion de version, workflow de revue par binôme

2. Architecture Technique

2.1 Vue d'ensemble

L'architecture repose sur le modèle acteur (Actor Model) fourni par Akka Typed. Chaque entité du système – train ou contrôleur – est représentée par un acteur autonome qui communique exclusivement via des messages asynchrones. Cette approche garantit

l'absence de mémoire partagée et offre une correspondance naturelle avec le modèle formel.

Architecture Acteurs (extension PSD)

```
[Train 1] ──Demande / Sortie──────► [SectionController] ◄──Demande / Sortie──────── [Train 2]
    │                                        │                                          │
    │──ArriveeQuai / DepartQuai──────► [QuaiController]    ◄──ArriveeQuai / DepartQuai──│
    │                                        │                                          │
    └──OuverturePortes / Fermeture──► [GestionnairePortes] ◄──OuverturePortes / Ferme──┘

Réponses asynchrones : Autorisation | Attente | PortesOuvertes | PortesFermees
```

Figure 1 : architecture des acteurs Akka Typed. Les flèches représentent les flux de messages typés. Chaque contrôleur joue le rôle d'arbitre exclusif sur sa ressource (canton, quai, portes).

2.2 Composants principaux

SectionController

Contrôle l'accès au canton partagé. Machine à états à deux comportements :

- etatLibre() : le canton est libre, le premier train à demander est autorisé immédiatement.
- etatOccupe(occupant, fileAttente) : un train occupe le canton. Les autres demandeurs sont enfilés dans une FIFO et reçoivent un message Attente.

La file FIFO garantit la vivacité : aucun risque de famine.

QuaiController

Acteur ajouté par l'extension PSD. Contrôle l'accès au quai avec la même logique FIFO que SectionController : un seul train autorisé à quai à la fois. Messages : ArriveeQuai (demande) / Autorisation ou Attente (réponse) / DepartQuai (libération).

GestionnairePortes

Acteur ajouté par l'extension PSD. Gère les portes palières (PSD) avec deux états : portesFermees / portesOuvertes. La sûreté PSD-Open est garantie de manière structurelle — par la pré-condition Petri Ti_a_quai et par la machine à états du Train qui n'émet OuverturePortes que depuis son comportement comportementAQuai. Le GestionnairePortes rejette en outre toute demande concurrente lorsque les portes sont déjà ouvertes. La sûreté PSD-Departure (les portes doivent être fermées avant le départ) est garantie par la pré-condition de la transition Petri Ti_depart_quai.

Train

Avec l'extension PSD, chaque acteur Train parcourt un cycle complet : demande d'accès au canton → entrée canton → arrivée quai → ouverture portes → fermeture portes → départ quai → nouveau cycle.

2.2 Protocole de messages (10 types)

Message	Émetteur	Destinataire	Signification
Demande(idTrain, repondreA)	Train	SectionController	Demande d'accès au canton
Autorisation	SectionController / QuaiController	Train	Accès accordé
Attente	SectionController	Train	Ressource occupé – file d'attente FIFO
Sortie(idTrain)	Train	SectionController	Libération du canton
ArruveeQuai(idTrain, replyTo)	Train	QuaiController	Demande d'accès au quai
DepartQuai(idTrain)	Train	QuaiController	Libération du quai
OuverturePortes(idTrain, replyTo)	Train	GestionnnairePortes	Demande d'ouverture des portes
FermeturePortes(idTrain, replyTo)	Train	GestionnairePortes	Demande de fermeture des portes
PortesOuvertes	GestionnairePortes	Train	Confirmation ouverture
PortesFermees	GestionnairePortes	Train	Confirmation fermeture

3. Modèle Formel – Réseau de Petri

3.1 Justification du choix du modèle

Le réseau de Petri permet de représenter naturellement les systèmes concurrents et les interactions entre ressources partagées. Il est particulièrement adapté aux systèmes critiques car il permet de raisonner sur l'ensemble des comportements possibles et pas seulement sur quelques exécutions.

Dans notre cas, il permet de modéliser précisément l'exclusion mutuelle entre les trains, ainsi que la synchronisation avec les ressources comme le canton, le quai et les portes.

Pour visualiser les schémas du réseau de Petri, se référer au dossier /Pétri.

3.2 Places du réseau (12)

4 ressources globales

Place	Signification	M0
Canton_libre	Le canton de signalisation est libre	1
Quai_libre	Le quai en station est libre	1
Portes_fermees	Les portes palières sont fermées	1
Portes_ouvertes	Les portes palières sont ouvertes	0

4 états par train (×2 = 8 places)

Place	Signification	M0 T1	M0 T2
Ti_hors	Train hors zone (en amont ou après avoir quitté le quai)	1	1
Ti_attente	Train a demandé l'accès au canton et attend	0	0
Ti_sur_canton	Train occupe le canton de signalisation	0	0
Ti_a_quai	Train à l'arrêt en station, sur le quai	0	0

3.3 Transitions du réseau (6)

Transition	Pré (consomme)	Post (produit)	Sens métier
T1_demande	T1_hors	T1_attente	Train 1 envoie une demande d'accès au canton
T1_entree_canton	T1_attente, Canton_libre	T1_sur_canton	Contrôleur autorise Train 1 à entrer dans le canton
T1_arrivee_quai	T1_sur_canton,Quai_libre	T1_a_quai,Canton_libre	Train 1 arrive à quai, libère le canton

T1_depart_quai	T1_a_quai, Portes_fermees	T1_hors, Quai_libre, Portes_fermees	Train 1 repart (portes fermées requises)
T2_demande	T2_hors	T2_attente	Train 2 envoie une demande d'accès au canton
T2_entree_canton	T2_attente, Canton_libre	T2_sur_canton	Contrôleur autorise Train 2 à entrer dans le canton
T2_arrivee_quai	T2_sur_canton, Quai_libre	T2_a_quai, Canton_libre	Train 2 arrive à quai, libère le canton
T2_depart_quai	T2_a_quai, Portes_fermees	T2_hors,Quai_libre,Portes_fermees	Train 2 repart (portes fermées requises)
Ouverture_portes_T1	Portes_fermees, T1_a_quai	Portes_ouvertes, T1_a_quai	Ouverture des portes pour Train 1
Ouverture_portes_T2	Portes_fermees, T2_a_quai	Portes_ouvertes, T2_a_quai	Ouverture des portes pour Train 2
Fermeture_portes_T1	Portes_ouvertes, T1_a_quai	Portes_fermees, T1_a_quai	Fermeture des portes par Train 1
Fermeture_portes_T2	Portes_ouvertes, T2_a_quai	Portes_fermees, T2_a_quai	Fermeture des portes par Train 2

3.4 Invariants du réseau

Le modèle étendu possède six invariants de marquage vérifiés exhaustivement par l'analyseur (champs de ResultatAnalyse dans Analyseur.scala) :

- (I1) Invariant canton : M(T1_sur_canton) + M(T2_sur_canton) + M(Canton_libre) \= 1 Garantit qu'à tout instant au plus un train occupe le canton.
- (I2) Invariant quai : M(T1_a_quai) + M(T2_a_quai) + M(Quai_libre) \= 1 Garantit qu'au plus un train est à quai à la fois.
- (I3) Invariant portes : M(Portes_fermees) + M(Portes_ouvertes) \= 1 Garantit que les portes sont soit ouvertes soit fermées (jamais les deux).
- (I4) Invariant par train : ∀i ∈ {1,2}, M(Ti_hors) + M(Ti_attente) + M(Ti_sur_canton) + M(Ti_a_quai) \= 1 Garantit qu'un train se trouve toujours dans exactement un état de son cycle de vie.

- (I5) PSD-Open Safety : M(Portes_ouvertes) \= 1 ⟹ M(T1_a_quai) + M(T2_a_quai) \= 1 Garantit qu'on n'ouvre les portes que si un train est présent à quai.

- (I6) PSD-Departure Safety : le tir de T1_depart_quai (resp. T2) requiert Portes_fermees \= 1, ce qui garantit qu'un train ne repart jamais portes ouvertes.

3.5 Vue d'ensemble du réseau

Le réseau regroupe les 12 places et 12 transitions décrites en *§3.2 Places du réseau* et *§3.3 Transitions du réseau*, organisées en trois sous-ensembles visuels :

- en bleu à droite, le cycle de vie du Train 1 (4 places, 4 transitions) ; en vert au centre, les ressources partagées (Canton_libre, Quai_libre, Portes_fermees, Portes_ouvertes)
- en rouge à gauche, le cycle de vie symétrique du Train 2.

Les arcs pleins sont des arcs Petri standards (consommation ou production de jetons). Les arcs en pointillés bleus bidirectionnels sont des read-arcs : la place est consommée puis immédiatement reproduite par la même transition (encodage maison, cf. docs preuves-manuelles.md §0). Ils représentent une condition testée sans modification d'état exactement la sémantique d'une garde de sûreté.

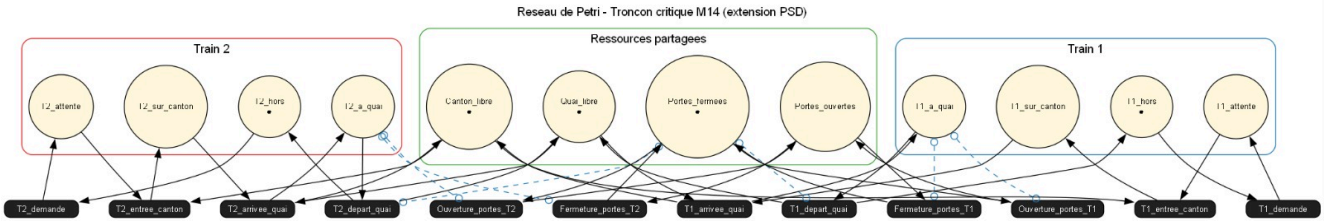


Figure 1 – Réseau de Petri complet : canton (T1/T2), quai, portes palières PSD

3.6 Zoom sur le sous-système des portes palières (PSD)

La vue d'ensemble étant dense, nous isolons ci-dessous le sous-réseau qui matérialise les sûretés PSD-Open et PSD-Departure. Il est composé des places Portes_fermees, Portes_ouvertes, Ti_a_quai, Ti_hors, Quai_libre et des six transitions qui les manipulent (Ouverture_portes_Ti, Fermeture_portes_Ti, Ti_depart_quai).

Le rôle des read-arcs y est plus lisible :

- Ti_a_quai est testée par les ouvertures et fermetures de portes (le train doit rester à quai pendant l'opération sans être consommé) ;
- Portes_fermees est testée par Ti_depart_quai (les portes doivent être fermées pour autoriser le départ, mais le départ ne modifie pas leur état).

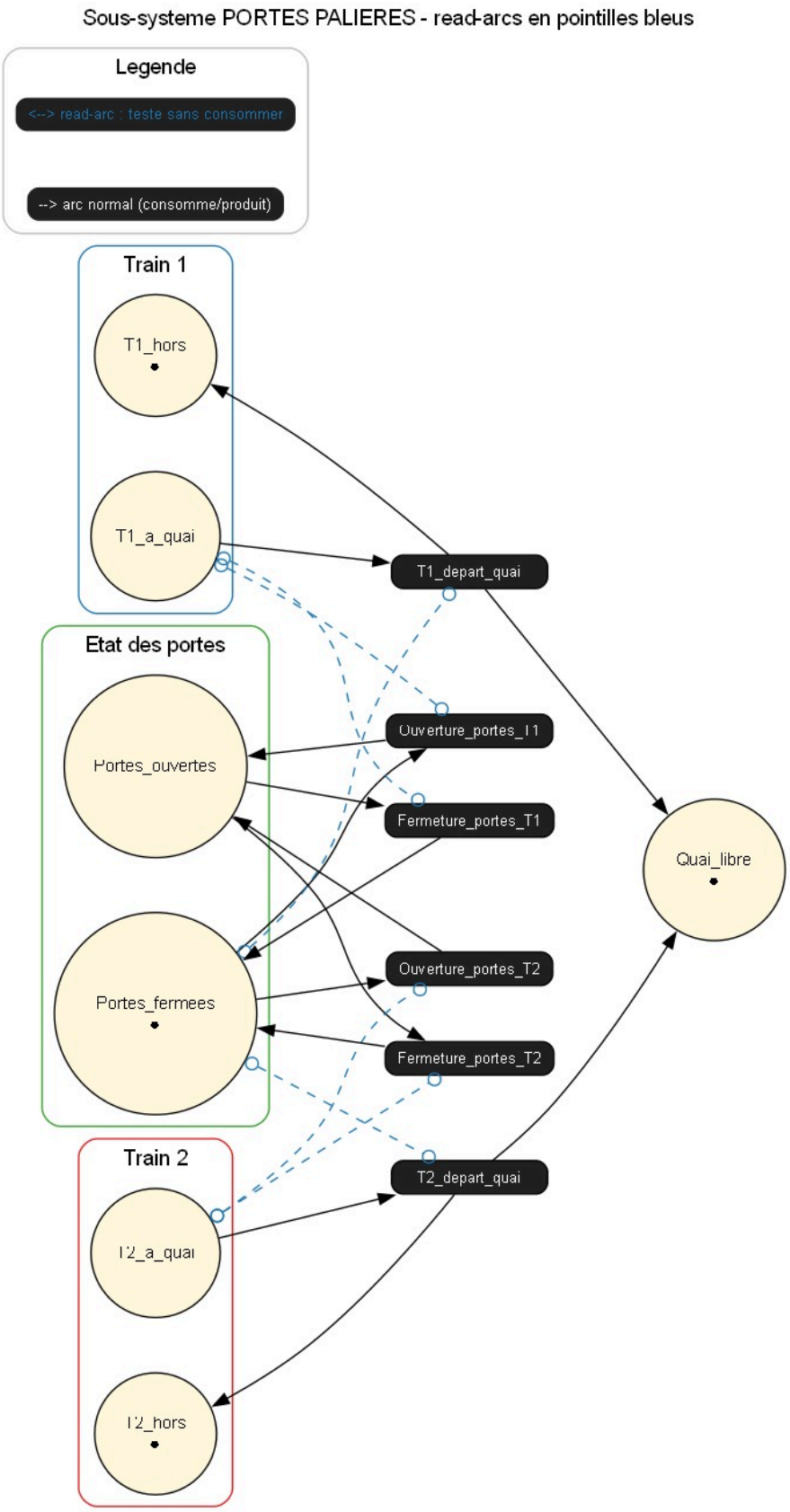


Figure 3 : zoom sur le sous-système PSD. L'unique jeton de l'invariant I3 circule entre Portes_fermees et Portes_ouvertes. Les read-arcs bleus matérialisent les pré-conditions PSD-Open (un train à quai) et PSD-Departure (portes fermées pour partir).

4. Vérification des Propriétés

4.1 Espace d'états atteignables

L'analyseur Scala effectue un BFS exhaustif depuis M0. Il produit exactement 20 marquages atteignables (M0..M19) et 40 arcs étiquetés. Le modèle initial (7 places) donnait 8 marquages ; l'extension PSD multiplie les états par la dimension des portes (ouvertes/fermées), générant 6 marquages supplémentaires avec Portes_ouvertes=1.

Index	Places marquées (M=1)	Trans. tirables	Successeurs
M0	Canton_libre, Quai_libre, Portes_fermees, T1_hors, T2_hors	T1_dem, T2_dem	M1, M2
M1	Canton_libre, Quai_libre, Portes_fermees, T1_attente, T2_hors	T1_ec, T2_dem	M3, M4
M2	Canton_libre,Quai_libre,Portes_fermees, T1_hors, T2_attente	T1_dem, T2_ec	M4, M5
M3	Quai_libre, Portes_fermees, T1_sur_canton, T2_hors	T1_aq, T2_dem	M6, M7
M4	Canton_libre, Quai_libre, Portes_fermees, T1_att, T2_att	T1_ec, T2_ec	M7,M8
M5	Quai_libre, Portes_fermees, T1_hors, T2_sur_canton	T1_dem, T2_aq	M8, M9
M6	Canton_libre, Portes_fermees, T1_a_quai, T2_hors	T1_dq, T2_dem, Ouv_T1	M0, M10, M11
M7..M10	(états intermédiaires – voir carnet docs/preuves-manuelles.md)	...	...
M11	Canton_libre, Portes_ouvertes, T1_a_quai, T2_hors	T2_dem, Ferm_T1	M6,M15
M13	Canton_libre, Portes_ouvertes, T1_hors, T2_a_quai	T1_dem,Ferm_T2	M9, M17
M18	Portes_ouvertes, T1_a_quai, T2_sur_canton	Ferm_T1	M14

M19	Portes_ouvertes, T1_sur_canton, T2_a_quai	Ferm_T2	M16

✓ **Résultats analyseur**

20 marquages atteignables • 40 arcs étiquetés • 6 invariants PASSE • 0 deadlock • 5 propriétés LTL PASSE

4.2 Propriétés de Sûreté (Safety)

Exclusion mutuelle canton
Sur chacun des 20 marquages atteignables, aucun ne contient simultanément T1_sur_canton=1 ET T2_sur_canton=1. L'invariantcanton (M(T1_sur_canton) + M(T2_sur_canton) + M(Canton_libre) = 1) est vérifié sur les 20 états par l'analyseur BFS.

Exclusion mutuelle quai
Sur chacun des 20 marquages atteignables, aucun ne contient simultanément T1_a_quai=1 ET T2_a_quai=1. L'invariant quai (M(T1_a_quai) + M(T2_a_quai) + M(Quai_libre) = 1) est vérifié sur les 20 états.

PSD-Open Safety
Sur tous les marquages atteignables : si Portes_ouvertes=1 alors exactement un train est à quai. Vérifié programmatiquement
(invariantPsdOpenOk = true).

PSD-Departure Safety
Sur tous les marquages atteignables : un train à quai ne peut partir que si Portes_fermees=1. Garanti structurellement par la précondition de T1_depart_quai et T2_depart_quai (invariantPsdDepartureOk = true).

| i | Conclusion Aucun marquage atteignable ne viole l'exclusion mutuelle du canton, ni du quai, ni les propriétés PSD-Open et PSD Departure Safety. Prouvé par énumération exhaustive BFS sur les 20 états. | | :—: | :—- |

4.3 Propriétés de Vivacité (Liveness)

Absence de deadlock : Pour chaque marquage atteignable M0–M19 (20 états), au moins une transition est tirable. L'analyseur confirme programmatiquement cette propriété (deadlocks = empty).

Progression après libération du canton : Sous hypothèse de fairness FIFO (garantie par l'implémentation Akka), tout train en attente (Ti_attente = 1) finit par accéder au canton. Vérifié par verifierGFLiveness sur le graphe d'accessibilité.

Progression PSD (T1_a_quai → F Portes_ouvertes) : Tout train arrivé à quai obtient éventuellement l'ouverture des portes. Vérifié

par verifierGFLiveness sur le graphe d'accessibilité.

4.4 Propriétés LTL

Propriété	Formule LTL	Statut
Sûreté – Exclusion mutuelle du canton	G ¬(T1_sur_canton ∧ T2_sur_canton)	Prouvée
Sûreté – Exclusion mutuelle du quai	G ¬(T1_a_quai ∧ T2_a_quai)	Prouvée
Sûreté – PSD-Open Safety	G (Portes_ouvertes → (T1_a_quai ⊕ T2_a_quai))	Prouvée
Vivacité – Progression canton T1	G (T1_attente → F T1_sur_canton)	Prouvée (sous fairness FIFO)
Vivacité – Progression canton T2	G (T2_attente → F T2_sur_canton)	Prouvée (sous fairness FIFO)
Vivacité - PSD T1	G (T1_a_quai → F Portes_ouvertes)	Prouvée
Bornitude	Tous les marquages atteignables ont des places ∈ {0,1}	Vérifiée (20 états finis)

5. Répartition des Tâches

5.1 Organisation générale

Le projet a été mené en équipe de quatre personnes (Duy, Alice, Axel, Ostrean) avec une organisation principalement individuelle, chacun avançant sur un domaine défini selon ses disponibilités et ses affinités techniques. Cette organisation n'était pas seulement un choix pratique : elle répondait aussi à une réalité humaine. Plusieurs membres de l'équipe ont rencontré, pendant la période du projet, des difficultés personnelles importantes (contraintes familiales, santé, situations de vie privée), ce qui a rendu le travail collectif continu plus compliqué que prévu.

Nous avons donc privilégié une répartition claire des responsabilités, avec des points de synchronisation ponctuels pour intégrer les contributions, vérifier la cohérence globale et prendre les décisions structurantes.

- ★ **Duy, Architecture & cœur formel**

  Initialisation du projet sbt, protocole Akka (Protocol, Train, SectionController), extension PSD (QuaiController, GestionnairePortes, Main), analyseur Petri BFS, scénarios de traces, README de livraison.

- ★ **Alice, Tests & validation**

  Tests ScalaTest des acteurs (Train, SectionController, QuaiController, GestionnairePortes), tests des invariants de l'analyseur Petri, vérification de l'exclusion mutuelle et du jeton portes.

★ **Axel, Simulation & visualisation**

Simulateur web HTML/JS, traces JSON nominales et critiques, traces de concurrence entre les deux trains.

★ **Ostrean, Documentation & rendu final**

Rapport final, bibliographie commentée, carnet de preuves manuelles, mise en cohérence des livrables, dernier push.

Les choix structurants (PSD, format des traces JSON, organisation des preuves manuelles, doctrine simple-propre-vérifiable) ont été discutés collectivement lors des points de synchronisation. Nous avons fait le choix de ne pas formaliser de « carnet de pourcentages » par membre : sur un projet aussi imbriqué, et compte tenu du contexte humain de l'équipe, une attribution chiffrée serait artificielle.

6. Tests & Validation

6.1 Stratégie de test

La stratégie de validation repose sur deux niveaux complémentaires : les tests d'intégration Akka (comportements concurrents) et la vérification formelle par l'analyseur Petri. Les deux méthodes sont conçues pour se confirmer mutuellement.

6.2 Résultats des tests

Suite de tests	Fichier	Tests	Résultat
SectionController	SectionControllerSpec.scala	3 cas	3/3 verts
Train (extension PSD)	TrainSpec.scala	6 cas	6/6 verts
QuaiController	QuaiControllerSpec.scala	5 cas	5/5 verts
GestionnairePortes	GestionnairePortesSpec.scala	5 cas	5/5 verts
Analyseur Petri	AnalyseurSpec.scala	30 cas	30/30 verts
TOTAL	—	49 cas	49/49 verts

6.3 Scénarios critiques couverts

Trois scénarios traduisent ligne par ligne les messages Akka en transitions Petri (livrable L6, voir docs/comparaison-akka petri.md). Chaque trace exécutée est sérialisée en JSON dans le dossier demo/.

Scénario 1 – Cycle nominal complet (un train)

Un train réalise le cycle complet : Demande → Autorisation → ArriveeQuai → OuverturePortes → FermeturePortes →DepartQuai. Couvre 6 transitions Petri (T1_demande, T1_entree_canton, T1_arrivee_quai, Ouverture_portes_T1, Fermeture_portes_T1, T1_depart_quai). Marquage final = M0 (cycle fermé). Trace : demo/trace-nominal.json.

Scénario 2 – Concurrence canton + quai
Train 1 occupe le canton, Train 2 demande et reçoit Attente, puis Train 1 libère le canton en arrivant au quai. Train 2 est promu et entre dans le canton pendant que Train 1 ouvre les portes à quai. Vérifie l'indépendance des deux ressources (canton et quai) et la vivacité FIFO. Trace : demo/trace-concurrence.json et demo/trace-cycle-deux-trains.json.

Scénario 3 – Sûreté PSD (tentative invalide rejetée)
Tentative d'ouverture des portes alors qu'aucun train n'est à quai. Côté Akka : la machine d'états du Train interdit l'émission de OuverturePortes hors de l'état a_quai ; GestionnairePortes rejette toute demande d'un non-occupant (test PSD-Open Safety). Côté Petri : la transition Ouverture_portes_Ti n'est jamais tirable depuis un marquage où Ti_a_quai = 0. La preuve par contre-exemple confirme PSD-Open et PSD-Departure sur les 20 marquages atteignables. Trace : demo/trace violation.json et demo/trace-violation-depart.json.

7. Résultats Obtenus

7.1 Livrables produits

Livrable	Fichier	Statut
L1 – Bibliographie commentée	docs/biblio.md	Livré
L2 – Code Akka/Scala du tronçon	src/main/scala/m14/troncon/	Livré & testé
L3 – Réseau de Petri formel	petri	Livré
L4 – Rapport de vérification	rapport.md	Livré
L5 – Analyseur Petri en Scala	src/main/scala/m14/petri/	Livré
L6 – Comparaison Akka vs Petri	docscomparaison.md	Livré
L7 – Dépôt GitHub propre	https://github.com/MonsieurNikko/M14-CriticalSystem	Livré & tracé

7.2 Bilan technique

Le projet aboutit à une simulation fonctionnelle et cohérente démontrant la maîtrise des concepts de concurrence et de sûreté logicielle. Les 49 tests passent au vert. L'analyseur Petri confirme programmatiquement les preuves manuelles : 20 marquages atteignables, exclusion mutuelle canton et quai vérifiées, PSD-Open Safety et PSD-Departure Safety vérifiées, absence de deadlock prouvée, 40 arcs dans le graphe d'accessibilité.

7.3 Reproduction des résultats

Les résultats du projet sont entièrement reproductibles à partir de trois commandes exécutées depuis la racine du dépôt.

La première commande `sbt test` permet d'exécuter l'ensemble des tests ScalaTest. Elle lance 49 tests répartis sur 5 suites, tous validés sans erreur.

La deuxième commande `sbt "runMain m14.petri.Analyseur"` exécute l'analyse formelle du réseau de Petri. Elle réalise une exploration BFS complète de l'espace d'états, identifie les 20 marquages atteignables et vérifie automatiquement les 6 invariants ainsi que les 5 propriétés LTL. Le résultat confirme l'absence de deadlock et la validation de toutes les propriétés.

Enfin, l'ouverture du fichier `demo/index.html` dans un navigateur permet d'accéder à un démonstrateur interactif. Celui-ci rejoue les différentes traces du système, notamment les scénarios nominaux, de concurrence, et de violation, ainsi que les cycles impliquant deux trains.

Prérequis : JDK 11+, sbt 1.9+. Aucun outil tiers de réseaux de Petri n'est utilisé (conformément au cahier des charges § 2.1).

8. Conclusion & Perspectives

8.1 Bilan global

M14-CriticalSystem a permis de mettre en pratique des notions avancées de développement logiciel et de systèmes distribués : du codage en Scala, modèle acteur avec Akka, vérification formelle par réseaux de Petri, preuve d'invariants et analyse d'espaces d'états. Le projet répond à l'ensemble des objectifs initiaux et constitue une base solide, documentée et testée.

Les deux approches, simulation distribuée Akka et modèle formel Petri, ont été menées en parallèle et en cohérence, offrant une double garantie sur les propriétés critiques du système. Ce type de démarche est directement transposable aux systèmes industriels réels.

8.2 Limites assumées

- Pas de modélisation de pannes ou de timeouts (hypothèse de communications fiables).
- Fairness dépendante de l'implémentation Akka, non garantie par le modèle Petri pur.
- Comparaison qualitative entre Akka et Petri, non quantitative.
- Périmètre volontairement restreint à 2 trains et 1 tronçon.
- Pas de model checker complet : vérification structurelle + LTL informelle sur espace d'états fini.

8.3 Autocritique et limites de fond

Au-delà des limites « assumées par construction » précédentes, plusieurs limites de fond doivent être reconnues honnêtement :

- **Modèle très simplifié vs. réalité industrielle.** La ligne 14 réelle compte 21 stations, deux voies, des terminus avec retournement, plusieurs PCC (postes de commande

centralisés) et des procédures dégradées. Notre modèle tient sur 12 places et 12 transitions : il capture correctement le noyau (exclusion mutuelle, PSD) mais ne prétend pas modéliser la ligne entière. Pas de dimension temporelle quantitative. Les réseaux de Petri utilisés ici sont non temporisés. Il serait impossible, avec ce modèle, de vérifier des bornes du type « les portes restent ouvertes au plus 30 secondes ». Il faudrait passer à des Time Petri Nets (Berthomieu – Diaz) ou à TLA+ / UPPAAL pour ce type de propriété.

- **Akka/Scala** n'est pas une plateforme certifiée SIL4. La norme EN 50128 (logiciel ferroviaire critique) impose des langages et des outils homologués (typiquement Ada SPARK, C MISRA, ou génération de code certifiée depuis SCADE). Notre simulation Akka est pédagogique : un système réel ne pourrait pas reposer dessus en l'état.

- **Espace d'états minuscule** (20 marquages). L'énumération BFS exhaustive est triviale ici, ce qui est rassurant mais aussi peu représentatif des défis réels : sur N trains, l'espace croît exponentiellement et nécessite des techniques de réduction par symétries, d'ordres partiels ou d'abstraction que nous n'avons pas mises en œuvre.

- **Tests classiques mais pas de garanties statistiques fortes.** Les 49 tests ScalaTest valident des scénarios choisis. Aucun property-based testing (ScalaCheck) ni mutation testing (Stryker, PIT) n'a été mis en place : la couverture qualitative est bonne, la robustesse face à des entrées non anticipées n'est pas mesurée.

- **LTL vérifiée sur le graphe d'accessibilité, pas par un model checker complet.** Notre analyseur évalue les formules LTL à la main, sur un graphe fini. Un vrai outil (NuSMV, SPIN) construirait un automate de Büchi pour la négation de la formule et chercherait un cycle accepteur — méthode plus rigoureuse, qui dépassait le périmètre du projet. Ces limites ne remettent pas en cause la validité du travail dans son cadre pédagogique, mais elles délimitent honnêtement la portée des résultats : nous avons construit une maquette formellement vérifiée d'un noyau d'exclusion mutuelle, pas un système embarquable.

8.4 Perspectives d'amélioration

Extension	Description	Complexité
N trains (généralisation)	Étendre le modèle à N trains concurrents avec arbitrage généralisé	Moyenne
Tolérance aux pannes	Supervision Akka avec redémarrage automatique des acteurs défaillants	Élevée
Model checker LTL complet	Intégration d'un outil tel que SPIN ou NuSMV pour vérification automatisée	Élevée
Modèle temporisé	Passage à des réseaux de Petri temporels ou TLA+ pour les contraintes temps-réel	Élevée
Interface graphique	Visualisation en temps réel de la simulation et de l'état du réseau de Petri	Faible
Réseau ferroviaire complet	Extension à plusieurs tronçons et gestion de l'ensemble de la ligne M14	Très élevée

9. Bibliographie (Livrable L1)

Bibliographie commentée complète disponible dans docs/bibliographie.md. Extrait des sources structurantes :

- **Réseaux de Petri**

Murata, T. (1989). Petri Nets: Properties, Analysis and Applications. Proceedings of the IEEE, 77(4), 541–580. doi.org 10.1109/5.24143. → Définitions formelles, P-invariants utilisés en section 3.4.

David, R. & Alla, H. (2010). Discrete, Continuous, and Hybrid Petri Nets (2e éd.). Springer. → Justification de l'analyse par exploration BFS de l'espace d'états (livrable L5).

- **Modèle acteur & Akka**

Hewitt, C., Bishop, P., Steiger, R. (1973). A Universal Modular ACTOR Formalism for Artificial Intelligence. IJCAI. → Source théorique du modèle acteur.

Lightbend (2024). Akka Typed Actors documentation. doc.akka.io/docs/akka/current/typed/. → Référence d'usage du framework (Behavior[T], TestKit).

- **Vérification formelle & LTL**

Baier, C. & Katoen, J.-P. (2008). Principles of Model Checking. MIT Press. → Formalisation des propriétés LTL Safety / Liveness (section 4.4).

Manna, Z. & Pnueli, A. (1995). Temporal Verification of Reactive Systems: Safety. Springer. → Distinction Safety / Liveness, structure de la section 4.

Pnueli, A. (1977). The temporal logic of programs. Proc. 18th IEEE FOCS, 46–57. → Source originelle de la LTL.

- **Concurrence & systèmes distribués**

Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM, 21(7), 558 565. doi.org/10.1145/359545.359563. → Justification de l'arbitre centralisé (SectionController, QuaiController).

Magee, J. & Kramer, J. (2006). Concurrency: State Models and Java Programs (2e éd.). Wiley. → Inspiration du découpage en machines à états des acteurs Train.

- **Domaine ferroviaire critique (M14 / CBTC / PSD)**

IEEE Std 1474.1-2004. Standard for Communications-Based Train Control (CBTC) Performance and Functional Requirements. →Justification du scope (block protection, exclusion mutuelle canton).

RATP. Documentation publique de la ligne 14. ratp.fr. → Contexte applicatif du métro automatique sans conducteur.

UITP – Union Internationale des Transports Publics. Platform Screen Doors and Mass Transit Safety. uitp.org. → Justification de la criticité réglementaire des invariants PSD-Open et PSD-Departure (section 3.4).